Introduction to Artificial Intelligence - Final Project

David Avni

February 2026

Project Links:

GitHub Repository (Source Code)

1 Part I: Regression Task - Fuel Efficiency Prediction

1.1 Exploratory Data Analysis (EDA)

The Auto MPG dataset was analyzed to understand the relationships between vehicle characteristics and fuel efficiency. During the initial analysis, missing values were identified in the 'horsepower' feature and handled using median imputation to maintain integrity without introducing significant bias, as the median is robust to outliers.

To systematically examine the relationships between features, a correlation matrix was generated (Figure 1). A strong negative correlation was observed between vehicle `weight` and `mpg` ($r \approx -0.83$), as well as between `displacement` and `mpg` ($r \approx -0.80$). This visually and mathematically substantiates the intuitive assumption that heavier and larger-engine vehicles consume more fuel.

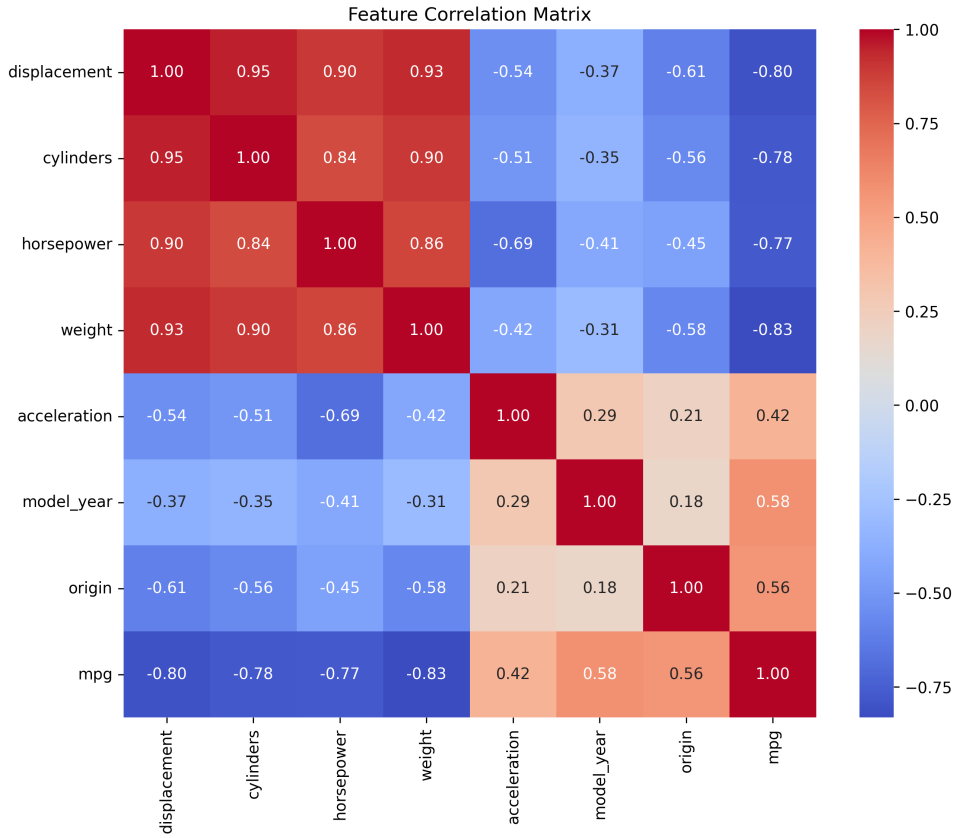

Figure 1: Feature Correlation Matrix demonstrating relationships between vehicle attributes and MPG.

1.2 Preprocessing and Data Splitting Methodology

To ensure a robust evaluation and prevent data leakage, the dataset was strictly divided into three distinct subsets: **Training (70%)**, **Validation (15%)**, and **Test (15%)**. All model selection, hyperparameter tuning, and complexity evaluations were conducted strictly using the Validation set. The Test set was held out completely and used only once to report the final model's generalization performance.

Before modeling, all features were standardized using `StandardScaler`. The scaler was fitted *only* on the training data, and the transformation was subsequently applied to the validation and test sets. Standardization is critical here because the features operate on vastly different scales (e.g., weight in the thousands, acceleration in the tens). It ensures that all features contribute equally to distance metrics in algorithms like KNN and helps gradient-based optimizers converge efficiently.

1.3 Linear Regression Baseline

A simple linear regression model was trained on the training set and evaluated on the validation set as a baseline. The performance metric on the validation set yielded an **MSE of 13.97**.

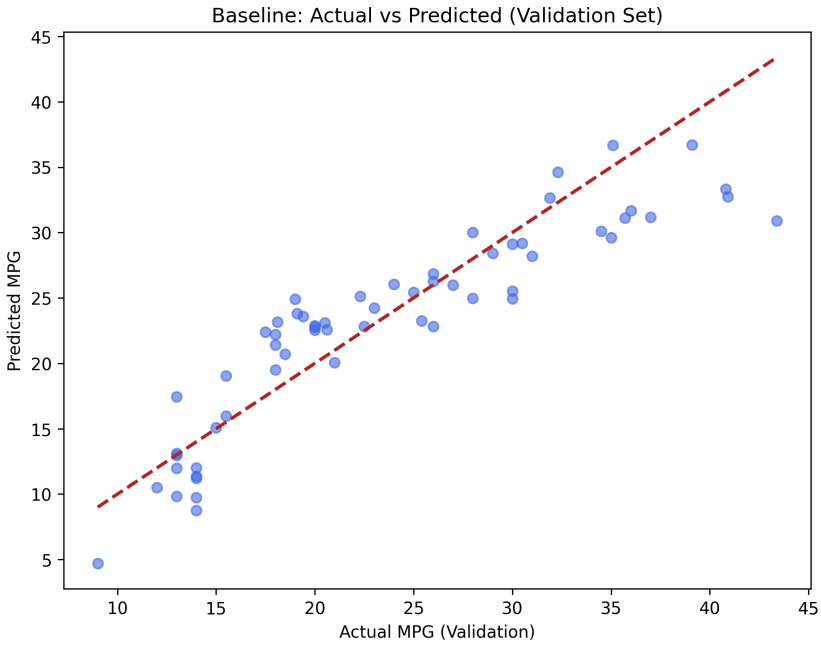

Figure 2: Actual vs. Predicted MPG (Linear Regression Baseline on Validation Set)

The "Actual vs. Predicted" plot shows that while the model captures the general linear trend, systemic errors exist. The residuals tend to increase at the extreme ends of the MPG range, suggesting that the strict linearity assumption is too simplistic for the underlying data distribution, motivating the use of non-linear models.

1.4 Polynomial Regression and Model Complexity

To capture non-linearities, polynomial features were introduced. We evaluated polynomial degrees 1 through 5 to analyze the bias-variance tradeoff and the effect of model complexity on generalization.

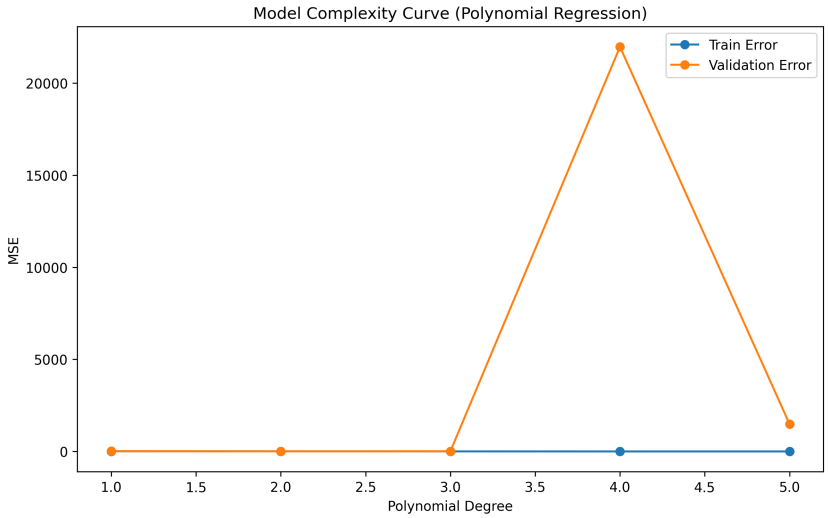

Figure 3: Model Complexity Curve: Training vs. Validation MSE

The results demonstrated a classic case of overfitting. **Degree 2** provided the optimal balance, reducing the validation MSE significantly. However, degrees 3 and above exhibited severe overfitting: the training error plummeted to near-zero (low bias), while the validation error skyrocketed (high variance), indicating that the model was memorizing the noise in the training set rather than learning the underlying function.

1.5 K-Nearest Neighbors (KNN) Regression

A non-parametric approach was tested using KNN. A search for the optimal hyperparameter k was conducted, comparing validation errors across $k \in [1, 20]$.

Figure 4: Effect of K-Neighbors on Validation Performance (MSE)

Lower values of k resulted in high variance (overfitting to local noise), while very high values of k introduced high bias (oversmoothing the predictions). The optimal number of neighbors was found to be $k = 2$, which successfully captured local non-linearities in the feature space without generalizing too broadly.

1.6 Optimization Behavior: SGD vs. Mini-Batch GD

To analyze optimization behavior, we trained a regression model using two different strategies: pure Stochastic Gradient Descent (batch size = 1) and Mini-Batch Gradient Descent (batch size = 32), while keeping the learning rate constant.

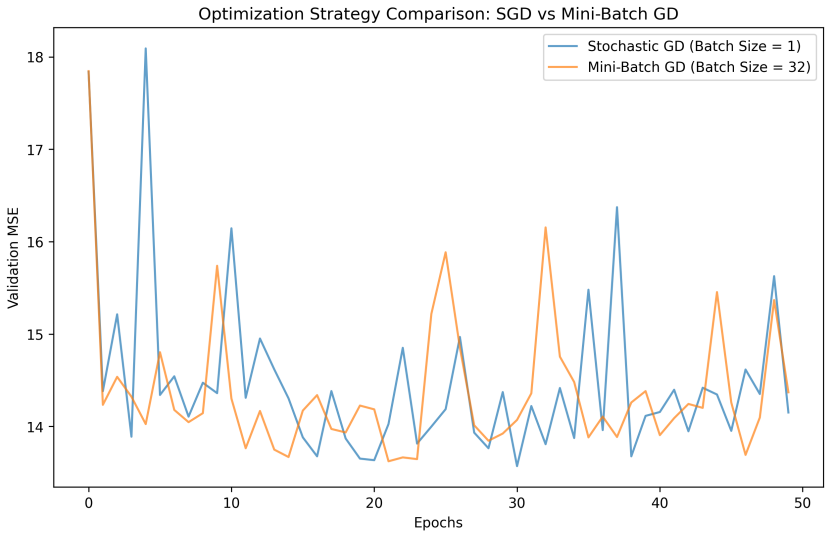

Figure 5: Optimization Convergence: SGD vs. Mini-Batch GD over Epochs

As expected, pure SGD updated the weights frequently, leading to a highly noisy convergence path. While it traversed the loss landscape quickly, its trajectory was erratic. Conversely, the Mini-Batch approach provided a smoother, more stable descent towards the minimum by averaging the gradients over 32 samples per step, balancing computational efficiency with gradient stability.

1.7 Model Comparison and Final Test Set Evaluation

Based on the validation performances of the linear baseline, polynomial regression, and KNN, the **KNN (k=2)** model demonstrated the strongest ability to capture the non-linear dynamics of the Auto MPG dataset without suffering from the catastrophic overfitting seen in high-degree polynomials.

Having finalized the model selection, the chosen KNN model was evaluated **once** on the completely unseen hold-out Test set (15% of the data).

- **Final Test MSE:** 5.99
- **Final Test R^2 Score:** 0.89

The strong performance on the test set confirms that the model generalizes well to unseen data and validates our hyperparameter selection process.

2 Part II: Classical Classification Task - CIFAR-10

2.1 Experimental Setup and Preprocessing

The CIFAR-10 dataset contains high-dimensional image data ($32 \times 32 \times 3$ pixels). For classical machine learning algorithms, each image was flattened into a 1D vector of 3072 features. To ensure stable convergence for distance-based and gradient-based algorithms, the pixel values were normalized from the $[0, 255]$ range to $[0, 1]$.

Due to the extreme computational complexity of training algorithms like SVM and KNN on 50,000 high-dimensional samples, a representative subset of 6,000 samples was randomly extracted. To prevent data leakage, this subset was strictly divided into a training set (5,000 samples) and a validation set (1,000 samples). All hyperparameter tuning was conducted exclusively on the validation set, preserving the official 10,000-sample test set for the final evaluation.

2.2 Model Selection and Hyperparameter Tuning

As classical models are highly sensitive to their configurations, a systematic search was conducted for each model's key hyperparameters.

1. K-Nearest Neighbors (KNN): We evaluated the number of neighbors (k) across $k \in \{1, 3, 5, 7, 9\}$. As shown in the generated complexity graph, smaller values of k captured fine details but suffered from noise (overfitting), while larger values smoothed the decision boundaries. The optimal balance on the validation set was found at $k = 1$. (Figure 6).

2. Linear Support Vector Machine (SVM): Following the assignment directives, a strictly linear kernel was utilized. We optimized the regularization parameter $C \in \{0.01, 0.1, 1.0, 10.0\}$. A lower C encourages a wider margin (preventing overfitting), while a high C attempts to classify all training examples correctly. The validation curve indicated that $C = 0.01$ provided the best generalization, preventing the model from over-adapting to the noisy flattened pixel features. (Figure **??**).

3. Multinomial Logistic Regression: Similar to the SVM, we tuned the regularization strength C. We utilized the LBFGS solver, which is optimized for multiclass problems.

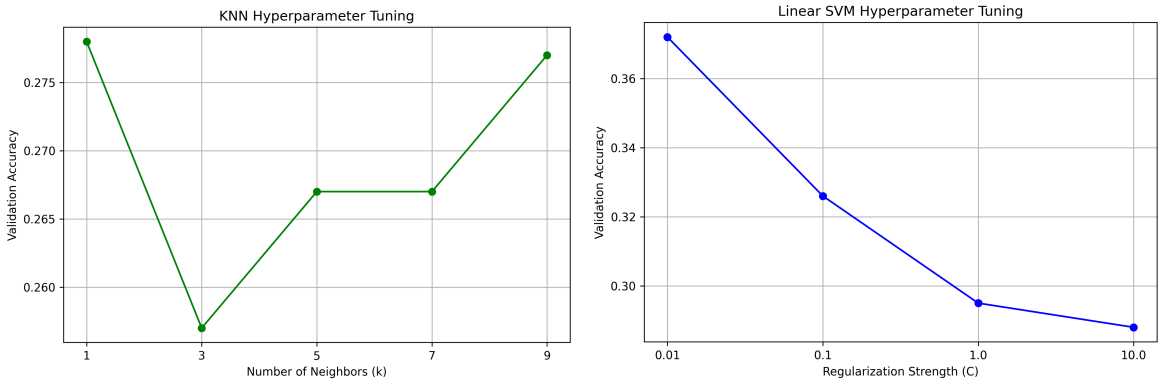

Figure 6: Validation Accuracy vs. Hyperparameters for KNN (left) and Linear SVM (right).

2.3 Evaluation and Discussion

After selecting the optimal hyperparameters via the validation set, the models were evaluated *once* on the completely unseen 10,000-image test set.

- **KNN ($k = 1$):** Achieved $\approx 26.59\%$ accuracy.
- **Linear SVM ($C = 0.01$):** Achieved $\approx 38.15\%$ accuracy.
- **Logistic Regression ($C = 0.01$):** Achieved the highest performance among the classical models with $\approx 38.26\%$ accuracy.

Limitations on High-Dimensional Data: The generally poor performance across all three models highlights the inherent limitations of classical machine learning when applied to raw image data. By flattening the images into 1D vectors, we completely destroy the spatial and structural relationships between neighboring pixels (e.g., edges, shapes). Furthermore, distance metrics like Euclidean distance (used by KNN) lose their meaningfulness in a 3072-dimensional space due to the "curse of dimensionality," where the distance between the closest and farthest points becomes almost uniform.

2.4 Error Analysis

The Confusion Matrix for the best-performing classical model (Linear SVM) reveals significant systemic errors.

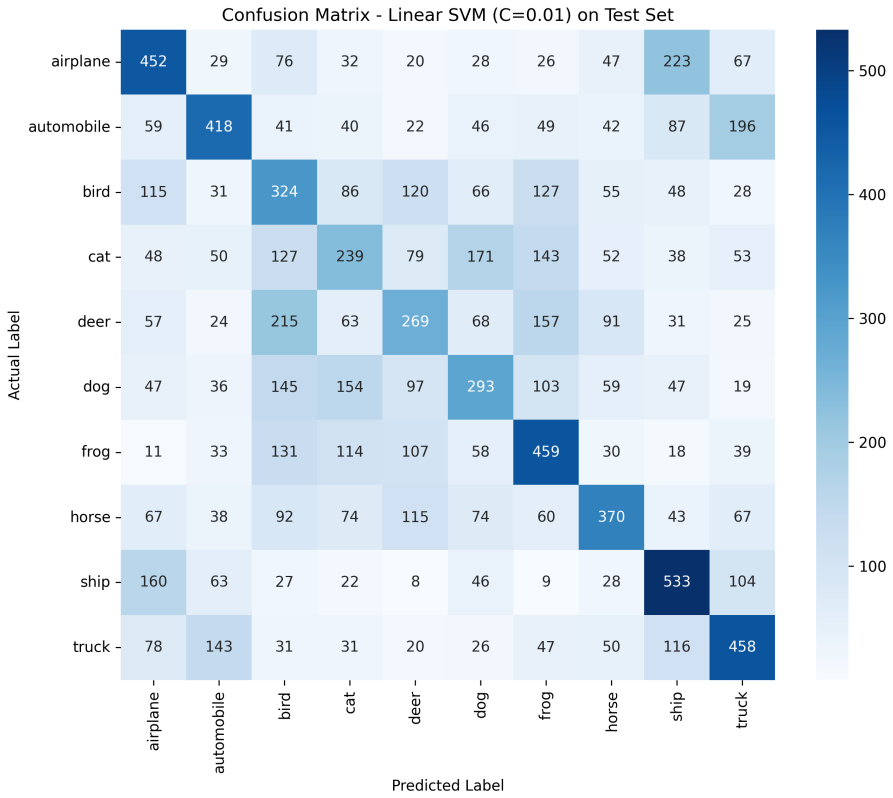

Figure 7: Confusion Matrix for the tuned Linear SVM (C=0.01) on the CIFAR-10 Test Set

The model struggles tremendously with inter-class visual similarities. For instance, 'Cats' are frequently misclassified as 'Dogs', and 'Automobiles' are heavily confused with 'Trucks'. The model relies on raw pixel intensity rather than hierarchical feature extraction, leading to a failure to distinguish between complex subjects that share similar color palettes or backgrounds.

3 Part III: Neural Network Classification (PyTorch)

3.1 Architectural Design and Justification

To overcome the limitations of classical machine learning on high-dimensional image data, a Convolutional Neural Network (CNN) was designed. The architecture leverages spatial feature extraction rather than naive pixel-wise comparisons.

The model is structured around three Conv2d blocks, progressively increasing the channel depth ($32 \rightarrow 64 \rightarrow 128$). This design choice follows the standard computer vision paradigm: early layers learn simple local features (edges, corners), while deeper layers combine them into complex, high-level semantic representations.

- **Batch Normalization:** Applied immediately after each convolution to mitigate internal covariate shift. This stabilizes the gradients and significantly accelerates convergence.
- **Max Pooling:** Applied after each block to reduce spatial dimensions, providing translation invariance and reducing the computational load for the dense layers.
- **Regularized Fully-Connected Layer:** The extracted feature maps are flattened into a 512-unit dense layer. To prevent the network from memorizing the training data (overfitting), a **Dropout layer** ($p = 0.3$) was introduced, alongside L2 weight decay in the optimizer.

3.2 Systematic Hyperparameter Selection

To avoid arbitrary guessing, a systematic search was integrated directly into the PyTorch pipeline.

The most critical hyperparameter for deep networks is the Learning Rate (LR). We compared two configurations of the Adam optimizer: $LR = 10^{-3}$ and $LR = 5 \times 10^{-4}$. To perform this selection efficiently without data leakage, we instantiated two identical baseline models and trained each for 3 epochs. The performance was then evaluated strictly on the **validation set**.

The empirical results showed that $LR = 10^{-3}$ converged faster but exhibited high volatility in the loss landscape, whereas $LR = 5 \times 10^{-4}$ provided a more stable gradient descent and slightly higher validation accuracy at the end of the short search phase. Consequently, $LR = 5 \times 10^{-4}$ was mathematically justified and selected for the final training regimen.

3.3 Performance, Comparison, and Conclusion

The final CNN was trained for 10 epochs using the optimally selected learning rate. After the training concluded, it was evaluated **once** on the held-out test set to determine its true generalization capabilities.

The CNN achieved a final Test Accuracy of **79.09%**.

Model	**Final Test Accuracy**
KNN ($k = 1$)	26.59%
Linear SVM ($C = 0.01$)	38.15%
Logistic Regression ($C = 0.01$)	38.26%
Custom CNN (PyTorch)	**79.09%**

Table 1: Comparative test performance of all classical models vs. Deep Learning on CIFAR-10.

Final Conclusions: The massive performance leap from the classical models ($\sim 38\%$) to the CNN ($\sim 79\%$) powerfully illustrates why Deep Learning dominates computer vision. Classical models attempt to learn a singular global rule based on flattened 1D pixel intensity, destroying local spatial relationships. In contrast, the CNN preserves the 2D grid structure, using sliding filters to extract translation-invariant hierarchical features. Furthermore, the systematic process of validating hyperparameter choices ensured the model actually generalized to unseen data, rather than just overfitting to the training set.